

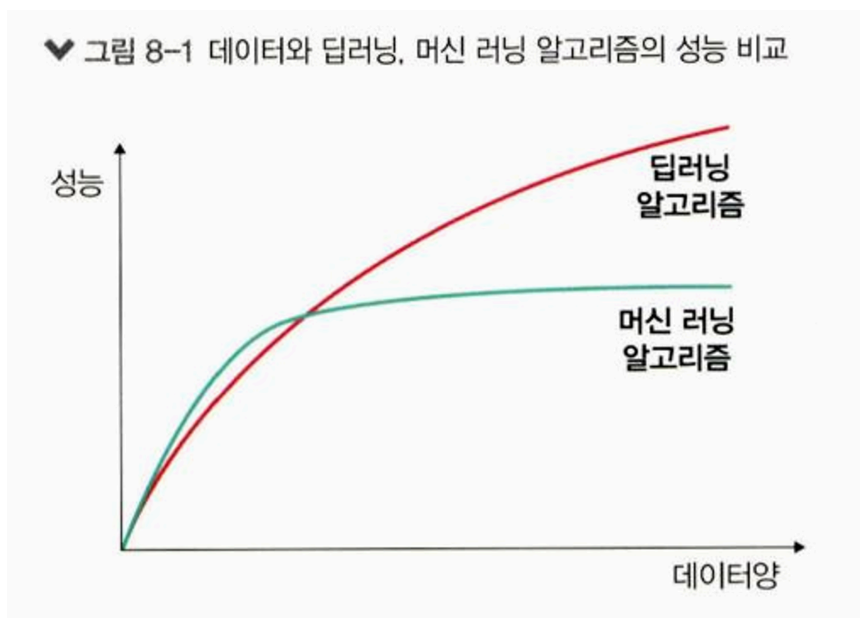
16주차 예습과제

🕒 과제 종류	예습과제
≡ 범위	딥러닝 파이토치 교과서 8장

Chapter8. 성능 최적화

8.1. 성능 최적화

8.1.1. 데이터를 사용한 성능 최적화

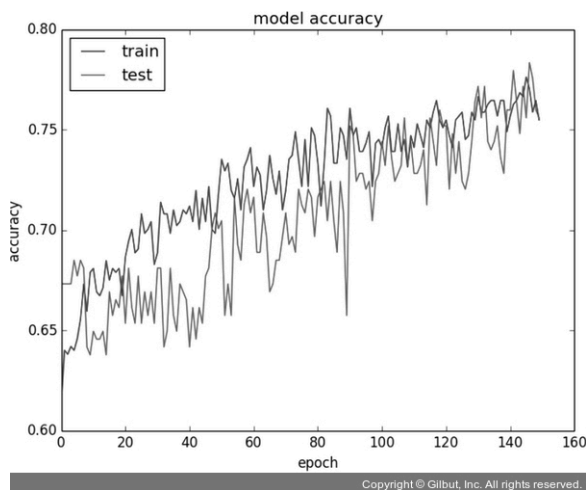


- ① 최대한 많은 데이터 수집하기: 데이터양이 많을수록 성능이 향상됨
- ② 데이터 생성하기: 전이학습과 같은 방법으로 만들어서 사용 가능. 이미지를 회전, 반전, 평행이동, 확대·축소하여 데이터를 늘릴 수 있음 (이미지 한정)
- ③ 데이터 범위 조정하기: 활성화 함수로 시그모이드나 하이퍼볼릭 탄젠트 등을 이용하여 조정할 수 있음
 - 정규화, 규제화, 표준화

8.1.2. 알고리즘을 이용한 성능 최적화

- 우리가 선택한 알고리즘이 최적이지 아닐 수 있으므로 모델을 훈련시켜 보고 최적의 성능을 보이는 알고리즘을 선택해야 함.
- 유사한 용도의 알고리즘
 - 시계열 데이터: RNN, LSTM, GRU 등
 - 데이터 분류: SVM, K-최근접 이웃 알고리즘

8.1.3. 알고리즘 튜닝을 위한 성능 최적화



- 하이퍼파라미터 튜닝
 - 다양한 하이퍼 파라미터를 변경하면서 훈련시키고 성능 비교가 필요
- 평가 결과를 바탕으로 모델 진단
 - 훈련(train) 성능이 검증(test)보다 눈에 띄게 좋다: 과적합(Overfitting) 의심
 - → 규제화를 진행한다면 성능 향상에 도움이 됨.
 - 훈련과 검증 성능 둘 다 좋지 않다: 과소적합(Underfitting) 의심
 - → 네트워크 구조 변경, 에포크 수 증가로 해결
 - 훈련 성능이 검증 성능을 넘어서는 변곡점이 있다 → Early Stopping 적용해보기
- **가중치**: 초깃값은 작은 난수를 사용함.
- **학습률**: 모델의 네트워크 구성에 따라 달라지므로, 초기에 크거나 작은 임의의 값을 선택한 후 학습 결과를 보며 조금씩 조정함.

- 네트워크 계층이 많으면 학습률을 크게, 계층이 적으면 학습률을 작게 설정
- **활성화 함수:** 활성화 함수를 변경하면 손실 함수도 함께 변경해야 하는 경우가 많으므로 신중하게 선택해야 함.
 - 일반적으로 시그모이드나 하이퍼볼릭 탄젠트를 사용했다면, 출력층에서는 소프트 맥스나 시그모이드 함수를 선택함.
- **배치와 에포크:** 큰 에포크와 작은 배치를 사용하는 것이 최근 딥러닝의 트렌드
- **옵티마이저 및 손실 함수:**
 - 일반적으로 확률적 경사 하강법(SGD)을 많이 사용함.
 - 네트워크 구성에 따라 다르지만, Adam이나 RMSProp도 좋은 성능을 보여줌.
- **네트워크 구성:** 최적의 네트워크를 찾는 것은 쉽지 않으므로, 구성을 변경해 가며 성능을 테스트해야 함.
 - 예를 들어, 하나의 은닉층에 뉴런을 여러 개 포함시키거나(넓은 네트워크), 계층을 늘리되 뉴런 개수는 줄이거나(깊은 네트워크), 두 가지를 결합하는 방법으로 최적의 네트워크를 확인한 후 결정함.

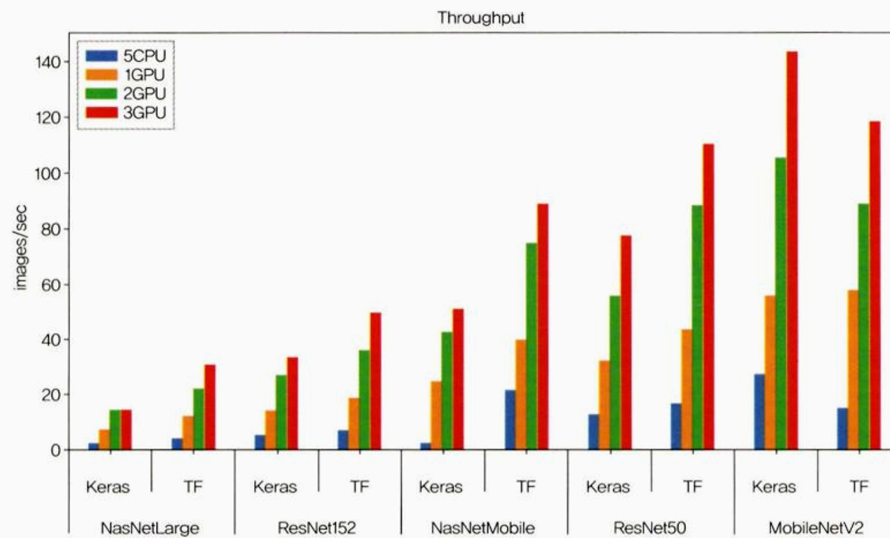
8.1.4. 앙상블을 이용한 성능 최적화

- **앙상블:** 모델을 두 개 이상 섞어서 사용하는 것

8.2. 하드웨어를 이용한 성능 최적화

8.2.1. CPU와 GPU 사용의 차이

♥ 그림 8-3 CPU와 GPU 성능 비교¹⁾



CPU

- 연산을 담당하는 ALU, 명령어를 해석하고 실행하는 control, 데이터를 저장하는 cache로 구성됨
- 명령어가 입력되는 순서대로 데이터를 처리하는 직렬 처리 방식
- 한 번에 하나의 명령어만 처리

GPU

- 병렬 처리를 위해 개발됨
- Cache 메모리 비중은 낮아지고, 연산을 수행하는 ALU 개수가 많아짐
- 개별 코어 속도는 CPU가 GPU보다 훨씬 빠름

8.2.2. GPU를 이용한 성능 최적화

CUDA는 NVIDIA에서 개발한 GPU 개발 툴.

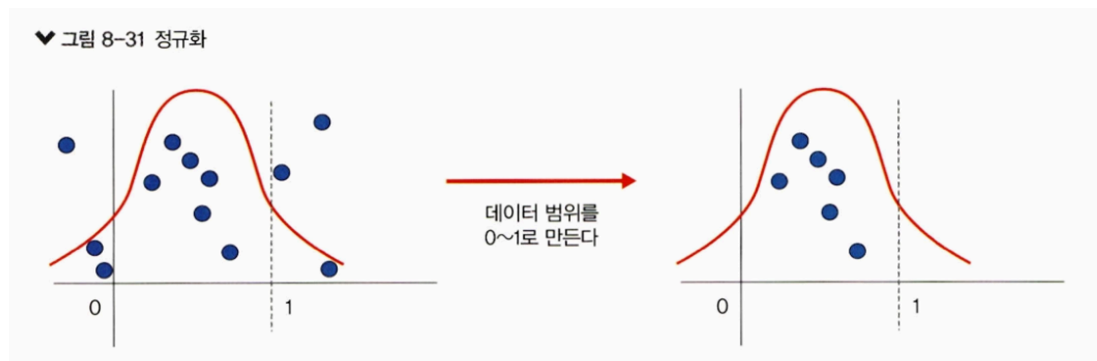
- CUDA를 사용하면 많은 양의 연산을 동시에 처리할 수 있다.
- CUDA가 등장한 이후 많은 프로그래머가 GPU를 이용한 프로그래밍을 할 수 있게 되었음.

8.3. 하이퍼파라미터를 이용한 성능 최적화

8.3.1. 배치 정규화를 이용한 성능 최적화

- 정규화

- 데이터 범위를 원하는 범위로 제한하는 것
- 이미지 데이터는 픽셀 정보로 0~255 사이의 값을 갖는데, 이를 255로 나누면 0~1.0 사이의 값을 가짐
- 특성 범위(scale)를 조정한다는 의미로 정규화를 특성 스케일링(feature scaling)이라고 부르기도 함.
- 스케일 조정을 위해 MinMaxScaler() 기법을 사용한다.

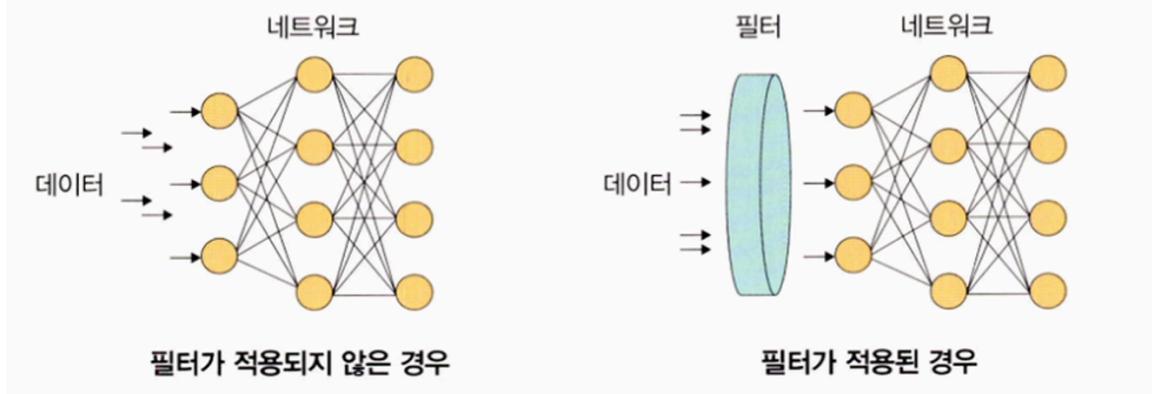


$$\frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

(x : 입력 데이터)

- 규제화

♥ 그림 8-32 규제화

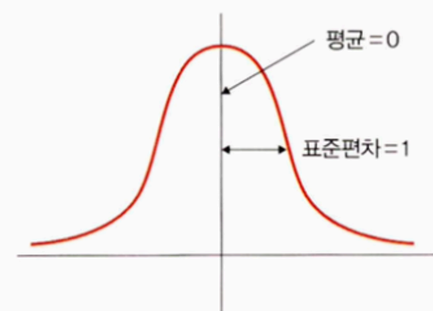


- 모델 복잡도를 줄이기 위해 제약을 두는 방법
- **제약:** 데이터가 네트워크에 들어가기 전에 필터를 적용한 것
- 드롭아웃, 조기 종료(early stopping rate) 등 존재
- 필터가 적용되지 않은 경우: 모든 데이터가 네트워크에 투입됨
- 필터가 적용된 경우: 걸러진 데이터만 네트워크에 투입 (빠르고 정확한 결과 얻을 수 있음)

• 표준화

- 기존 데이터를 평균은 0, 표준편차는 1인 형태의 데이터로 만드는 방법
- 다른 명칭으로 표준화 스칼라, z-스코어 정규화가 있음
- 데이터 분포가 가우시안 분포를 따를 때 유용하고, 평균을 기준으로 얼마나 떨어져 있는지 살펴볼 때 유용하게 쓰임.

♥ 그림 8-33 표준화

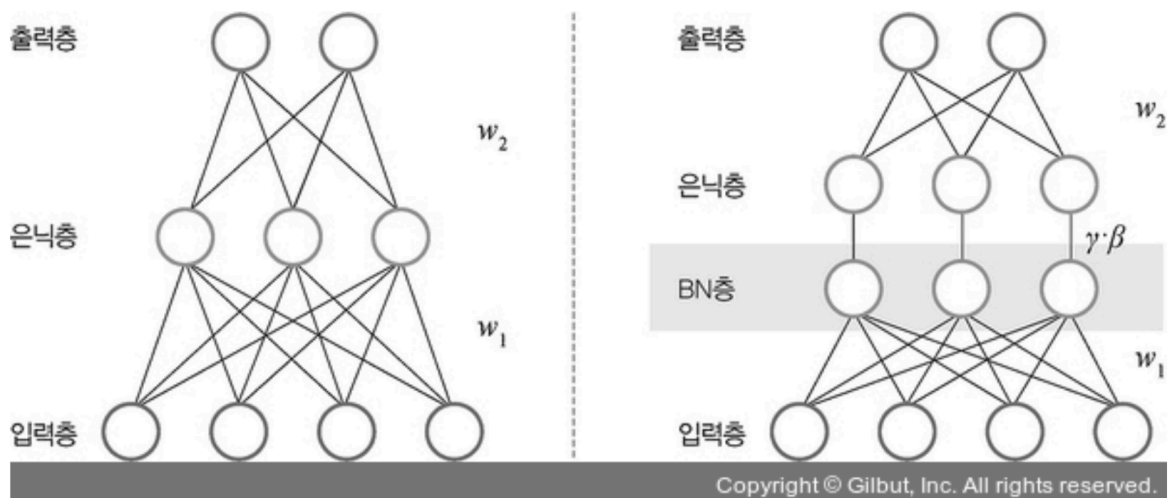


$$\frac{x - m}{\sigma}$$

(σ : 표준편차, x : 관측 값(입력 값), m : 평균)

• 배치 정규화

- 기울기 소멸(gradient vanishing), 기울기 폭발(gradient exploding)같은 문제를 해결하기 위한 방법
- 이 문제를 해결하기 위해 손실 함수로 렐루를 사용하거나 초깃값 튜닝, 학습률 조정
- 데이터 분포가 안정되어 학습 속도를 높일 수 있음
-

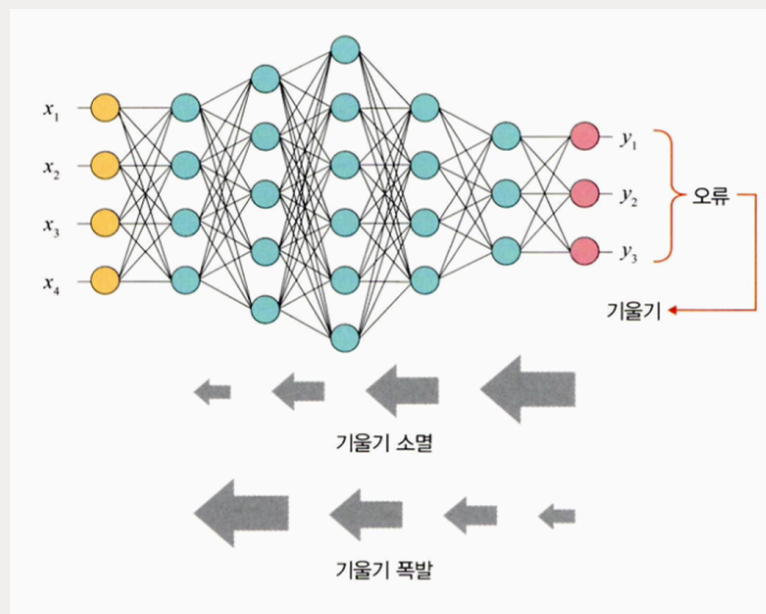


기울기 소멸, 기울기 폭발

- 기울기 소멸: 오차 정보를 역전파시키는 과정에서 기울기가 급격히 0에 가까워져 학습이 되지 않는 현상
- 기울기 폭발: 학습 과정에서 기울기가 급격히 커지는 현상

기울기 소멸과 폭발의 원인: 내부 공변량 변화(internal covariance shift)

- 내부 공변량 변화: 네트워크의 각 층마다 활성화 함수가 적용되면서 입력 값들의 분포가 계속 바뀌는 현상
- 이를 해결하기 위해 미니 배치(mini-batch)에 표준화와 유사한 방식을 적용하여 평균은 0, 표준편차는 1로 유지함



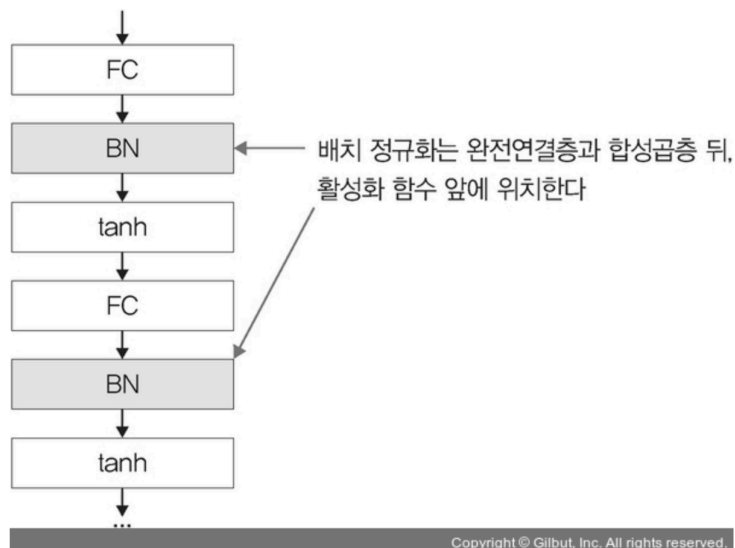
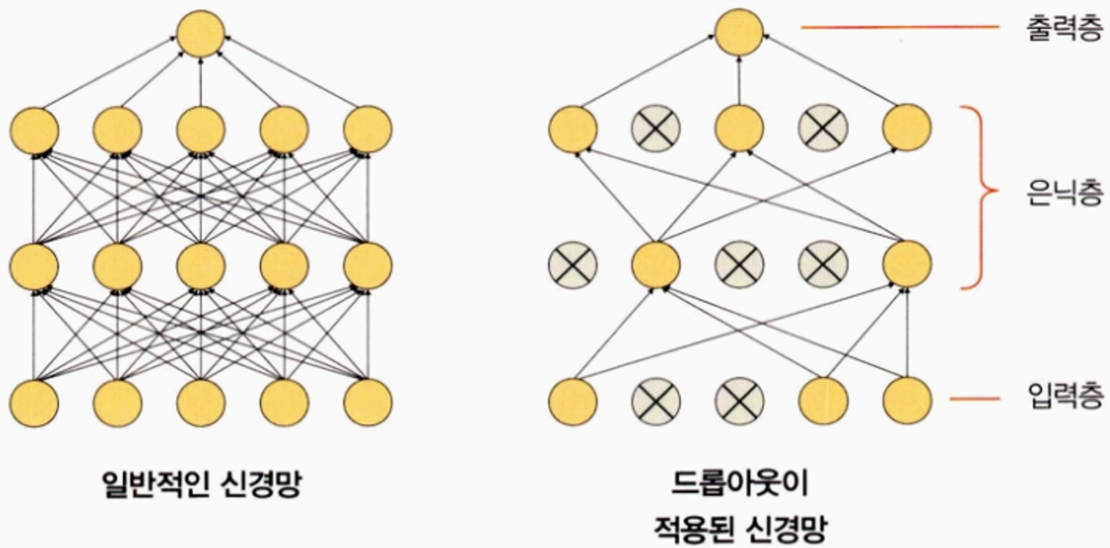
1. 미니 배치 평균을 구함
2. 미니 배치의 분산과 표준편차를 구함
3. 정규화 수행
4. 스케일 조정 (데이터 분포 조정)

- 배치 정규화의 장점: 매 단계마다 활성화 함수를 거치면서 데이터셋의 분포가 일정해짐. 속도 향상 가능
 - 배치 정규화의 단점
 - 배치 크기가 작을 때는 정규화 값이 기존 값과 다른 방향으로 훈련될 수 있음.
 - RNN은 네트워크 계층별로 미니 정규화를 적용해야 하기에 모델이 복잡해지면 비효율적일 수 있음.
- ⇒ 이러한 문제 해결을 위해 가중치 변경, 네트워크 구성 변경 등 필요하지만, 배치 정규화를 적용하면 적용하지 않았을 때보다 성능이 좋아지기 때문에 많이 사용된다.

8.3.2. 드롭아웃을 이용한 성능 최적화

- 과적합 예방을 위해 사용됨.
- **드롭아웃**: 훈련 시 일정 비율의 뉴런만 사용, 나머지 뉴런에 해당하는 가중치는 업데이트 하지 않는 방법
 - 매 단계마다 사용하지 않는 뉴런을 바꾸어가며 훈련
 - 은닉층에 배치된 노드 중 일부를 임의로 끄면서 학습해 지나친 학습을 방지한다.
- 드롭아웃 신경망
 - 일부 노드들은 비활성화되고 남은 노드들로 신호가 연결되는 신경망 형태
 - 어떤 노드를 비활성화 할지는 학습 때마다 무작위로 선정된다.
 - 테스트 데이터로 평가할 때는 노드들을 모두 사용하여 출력하되 노드 삭제 비율(드롭아웃 비율)을 곱해서 성능을 평가한다.

▼ 그림 8-37 드롭아웃



8.3.3. 조기종료를 이용한 성능 최적화

• 조기 종료

- 뉴럴 네트워크가 과적합을 회피하는 규제 기법
- 훈련 데이터와 별도로 검증 데이터를 준비하고, 매 에포크마다 검증 데이터에 대한 오차(validation loss)를 측정하여 모델의 종료 시점을 제어
- 과적합이 발생하면 훈련 데이터셋의 오차는 감소하지만 검증 데이터셋의 오차는 증가함. 조기 종료는 검증 데이터셋의 오차가 증가하는 시점에서 학습을 멈추도록 조정

- 학습 종료 시기를 결정할 뿐 최고의 성능을 갖는 모델을 보장하지는 않음

